Open in Colab

Intermediate Machine Learning: Assignment 4

Deadline

Assignment 4 is due Thursday, November 20 by 11:59pm. Late work will not be accepted as per the course policies (see the Syllabus and Course policies on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Submission

Submit your assignment as a pdf file on Gradescope, and as a notebook (.ipynb) on Canvas. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered. Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

To produce the .pdf, please convert to html and then print to pdf. (You may want to use your pdf print menu to scale the pages to be sure that cells are not truncated.) To convert to html, you can use this converter notebook.

Topics

- Graph kernels and Laplacians
- Graph neural networks
- Policy gradient algorithms
- Deep Q-learning

This assignment will also help to solidify your Python skills. In particular, problem 2 uses PyTorch, whereas many of our implementations have used Tensorflow.

Problem 1: Graph kernels (20 points)

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as img
import sklearn
import random
from numpy.linalg import inv
%matplotlib inline
```

In [ ]:
```
# Helper functions for third part of exercise

def rgb2gray(rgb):
    """Function to turn RGB images in greyscale images."""
    return np.dot(rgb[..., :3], [0.2989, 0.5870, 0.1140])


def grid_adj(rows, cols):
    """Function that creates the adjacency matrix of
    a grid graph with predefined amount of rows and columns."""
    M = np.zeros([rows*cols, rows*cols])
    for r in np.arange(rows):
        for c in np.arange(cols):
            i = r*cols + c
            if c > 0:
                M[i-1, i] = M[i, i-1] = 1
            if r > 0:
                M[i-cols, i] = M[i, i-cols] = 1
    return M
```

The graph Laplacian for a weighted graph on n nodes is defined as

$$L = D - W$$

where W is an $n \times n$ symmetric matrix of positive edge weights, with $W_{ij} = 0$ if (i, j) is not an edge in the graph, and D is the diagonal matrix with $D_{ii} = \sum_{j=1}^{n} W_{ij}$. This generalizes the definition of the Laplacian used in class, where all of the edge weights are one.

1. Show that L is a Mercer kernel, by showing that L is symmetric and positive-semidefinite.
2. In graph neural networks we define polynomial filters of the form

$$P = a_0 I + a_1 L + a_2 L^2 + \cdots a_d L^d$$

where L is the Laplacian and $a_0, \ldots, a_d$ are parameters, corresponding to the filter parameters in standard convolutional neural networks.

If each $a_i \geq 0$ is non-negative, show that P is also a Mercer kernel.

1.1

1.1

By definition, W is symmetric. $W^T = W$. D is diagonal. So D is also symmetric. $D^T = D$

Therefore. $L^T = (D-W)^T = D^T - W^T = D - W = L$ $\Rightarrow$ L is symmetric

For $\forall x \in \mathbb{R}^n$. $x^T L x = x^T (D-W) x = x^T D x - x^T W x = \sum_{i=1}^{n} x_i^2 D_{ii} - \sum_{i=1}^{n}\sum_{j=1}^{n} x_i x_j W_{ij}$

We know that $D_{ii} = \sum_{j=1}^{n} W_{ij}$ $\Rightarrow$ $\sum_{i=1}^{n} x_i^2 D_{ii} = \sum_{i=1}^{n} x_i^2 \sum_{j=1}^{n} W_{ij} = \sum_{i=1}^{n}\sum_{j=1}^{n} x_i^2 W_{ij}$

Because W is symmetric, we know that $W_{ij} = W_{ji}$. So $\sum_{i=1}^{n}\sum_{j=1}^{n} x_i^2 W_{ij} = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} W_{ij}(x_i^2 + x_j^2)$

Therefore, $x^T L x = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} W_{ij}(x_i^2 + x_j^2 - 2x_i x_j) = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} W_{ij}(x_i - x_j)^2 \geq 0$, which means L is positive-semidefinite.

1.2

1.2

Since L is symmetric and positive-semidefinite, it can be written as $L = U \Lambda U^T$, where $UU^T = I$ and Λ is diagonal

$\Lambda = \text{diag}(\lambda_1, \cdots, \lambda_n)$. $\lambda_i \geq 0$ for all i in $1, 2, \cdots, n$.

$L^K = (U \Lambda U^T)^K = U \Lambda^K U^T$, where $\Lambda^K = \text{diag}(\lambda_1^K, \lambda_2^K \cdots, \lambda_n^K)$. $\lambda_i^K \geq 0$ for all i in $1, 2, \cdots, n$

Therefore. L^K is also symmetric and positive-semidefinite. so for any $x \in \mathbb{R}^n$. $X^T L^K x \geq 0$.

$P^T = a_0 I^T + a_1 L^T + a_2 (L^2)^T + \cdots + a_d (L^d)^T = a_0 I + a_1 L + a_2 L^2 + \cdots + a_d L^d = P$ $\Rightarrow$ P is symmetric

for any $x \in \mathbb{R}^n$. $x^T P x = x^T (a_0 I + a_1 L + \cdots + a_d L^d) x = a_0 x^T I x + a_1 x^T L x + \cdots + a_d x^T L^d x \geq 0$. ($a_K \geq 0$ and $x^T L^K x \geq 0$ for any K in $1, \cdots n$)

So P is positive-semidefinite.

Therefore, P is also a Mercer kernel.

3. This polynomial filter has many applications. A handful of these applications are based on the fact that, given a graph with a signal x, the value of $x^T L x$ will be low in case the signal is smooth (i.e. smooth transitions of x between neighboring nodes). A large $x^T L x$ means that we have a rough graph signal (i.e. a lot of jumps in x between neighboring nodes).

   An intersting application that uses this property is the so-called image inpainting process, where an image is seen as grid graph. Image inpainting tries to restore a corrupted image by smoothing out the neighboring pixel values. In this problem we corrupt an image by turning off (i.e. making the pixel value equal to zero) a certain portion of the pixels. Your goal will be to restore the corrupted image and hence recreate the original image.

   First, let's corrupt an image by turning off a portion of the pixels. For this exercise, we choose to turn off 30% of the pixels. The result is shown below. Try to understand the code, as some variables might be interesting for your work.

   The image "Yale_Bulldogs.jpg" can be found in this GitHub repo: https://github.com/YData123/sds365-fa25/tree/main/assignments/assn4

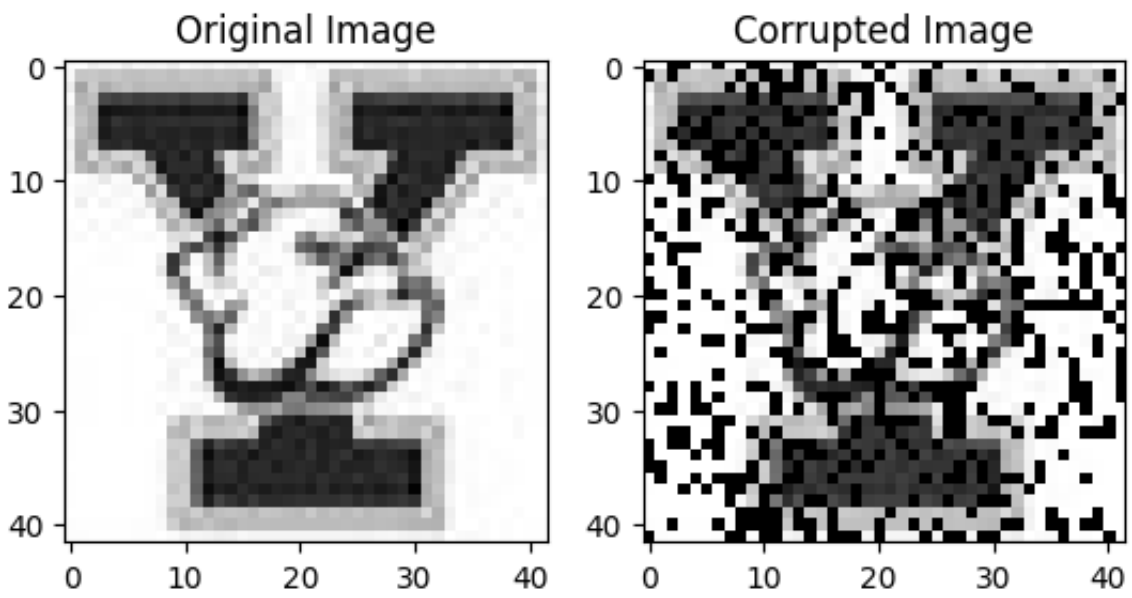
In [ ]:
```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [ ]:
```
# Normalize the pixels of the original image
image = img.imread("/content/drive/MyDrive/Yale_Bulldogs.jpg") / 255
# Turn picture into greyscale
gray_image = rgb2gray(image)
height_img = gray_image.shape[0]
width_img = gray_image.shape[1]

# Turn off (value 0) certain pixels
fraction_off = int(0.30*height_img*width_img)
mask = np.ones(height_img*width_img, dtype=int)
# Set the first fraction of pixels off
mask[:fraction_off] = 0
# Shuffle to create randomness
np.random.shuffle(mask)
# Multiply the original image by the reshapes mask
mask = np.reshape(mask, (height_img, width_img))
corrupted_image = np.multiply(mask, gray_image)

fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.imshow(gray_image, cmap=plt.get_cmap('gray'))
ax1.set_title("Original Image")
ax2.imshow(corrupted_image, cmap=plt.get_cmap('gray'))
ax2.set_title("Corrupted Image")
```

```
plt.show()
```



Inpainting missing pixel values can be formulated as the following optimization problem:

$$\min_{\mathbf{x}\in\mathbb{R}^n} \left\{ \|\mathbf{y} - \mathbf{M}\mathbf{x}\|_2^2 + \alpha \mathbf{x}^T \mathbf{P} \mathbf{x} \right\}$$

where $\mathbf{y} \in \mathbb{R}^n$ (n being the total amount of pixels) is the corrupted graph signal (with missing pixel values being 0) and α is a regularization (smoothing) parameter that controls for smoothness of the graph. $\mathbf{P}$ is the polynomial filter based on the laplacian $\mathbf{L}$. Finally, $\mathbf{M} \in \mathbb{R}^{n \times n}$ is a diagonal matrix that satisfies:

$$\mathbf{M}(i,i) = \begin{cases} 1, \text{ if } \mathbf{y}(\text{i}) \text{ is observed} \\ \\ 0, \text{ if } \mathbf{y}(\text{i}) \text{ is corrupted} \end{cases}$$

The optimization problem tries to find an $\mathbf{x}$ that matches the observed values in $\mathbf{y}$, and at the same time tries to be smooth on the graph. Start with deriving a closed form solution of this optimization problem:

1.3

We can see that M is symmetric, and $M^TM = M$

Let $f(x) = \|y - Mx\|^2 + \alpha x^T P x = (y-Mx)^T(y-Mx) + \alpha x^T P x = y^Ty - x^TMy - y^TMx + x^TMx + \alpha x^TPx$

differentiate on x and set the derivative to 0. $f'(x) = -2My + 2(M+\alpha P)x = 0 \Rightarrow x = (M+\alpha P)^{-1}My$

Next, let's restore our image. To keep things simple, let's say we already trained the polynomial filter $\mathbf{P}$ of degree 2 and we found the following weights:

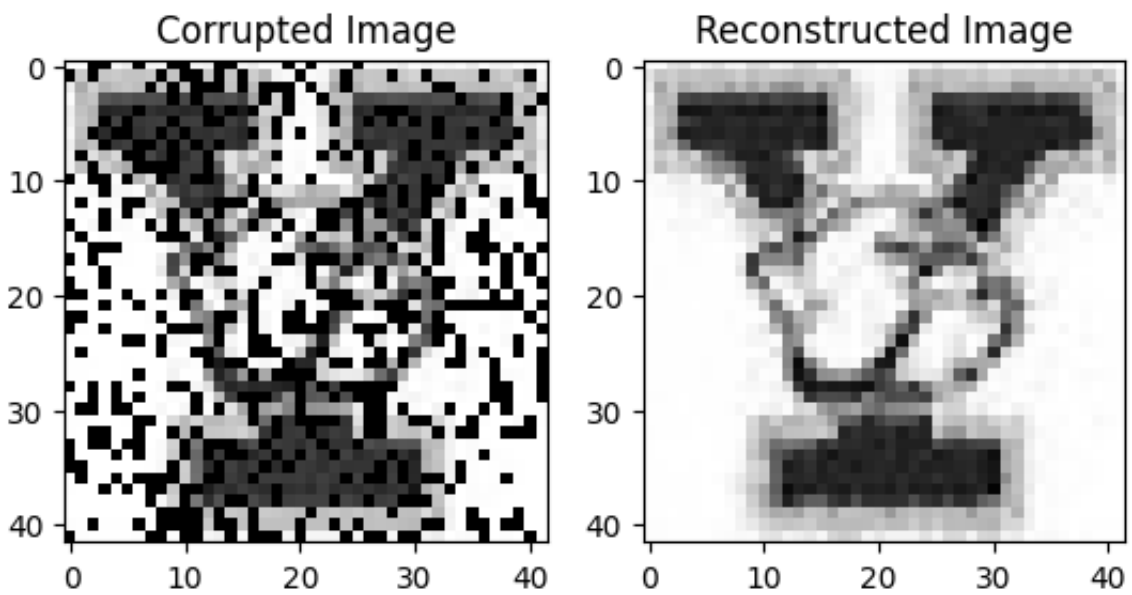
$$\mathbf{P} = \mathbf{L} + 0.05\,\mathbf{L}^2$$

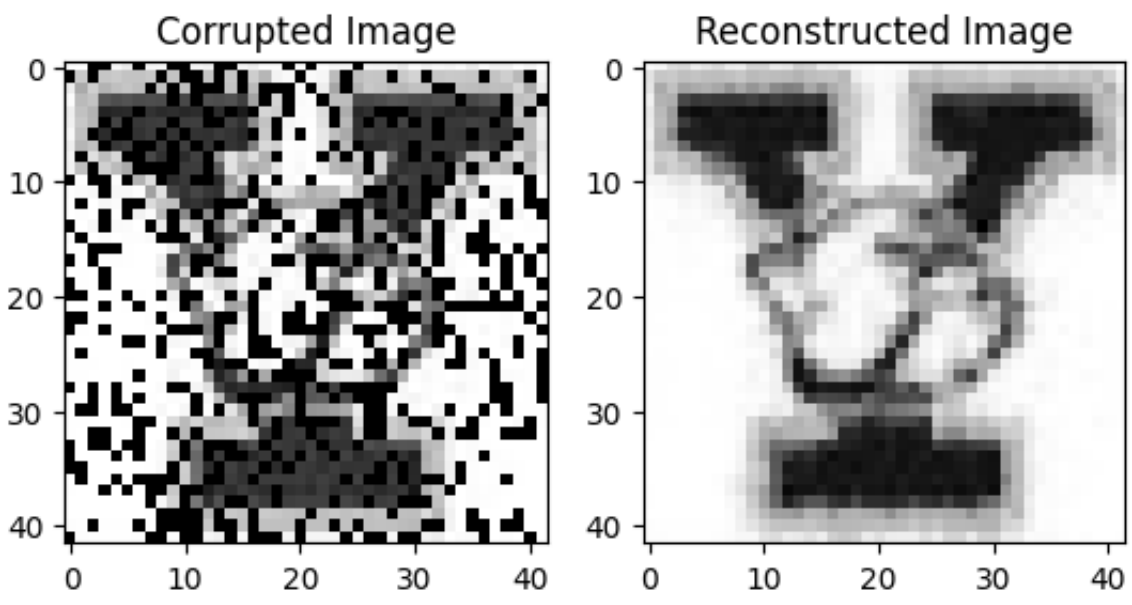
Fill in the following lines of code and show your reconstructed images next to the corrupted image. Assume that the weights on the graph edges are equal to 1.

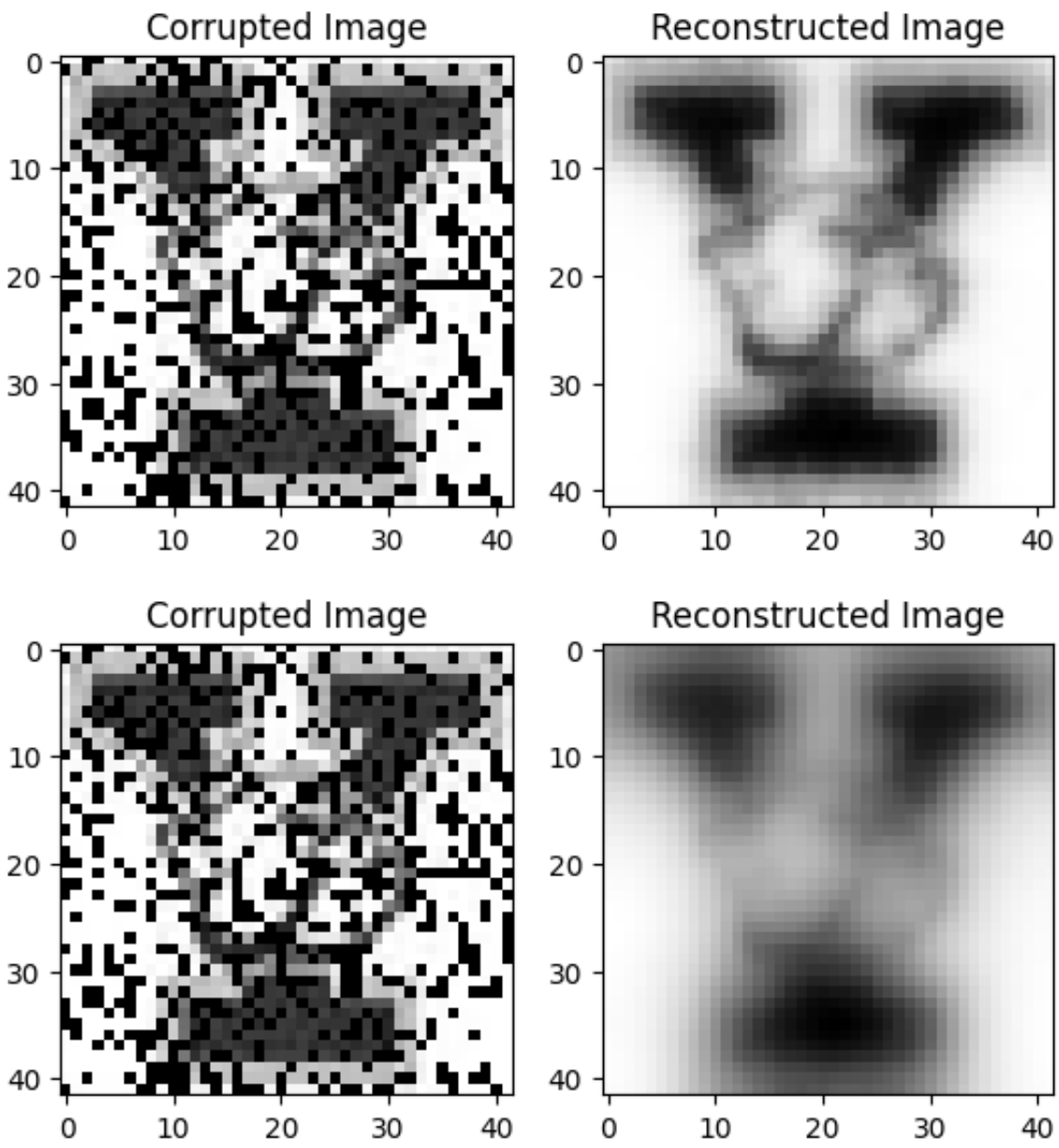
In [ ]:
```
# Corrupted graph signal
y = corrupted_image.reshape(height_img*width_img)

# Diagonal matrix defined as above
M = np.diag(mask.reshape(height_img*width_img))

# Adjacency matrix of the graph (by using the helper function)
A = grid_adj(height_img, width_img)

# Diagonal matrix defined as above
D = np.diag(np.sum(A, axis=1))

# Graph Laplacian defined as above
L = D - A

# Polynomial filter defined as above
P = L + 0.05 * np.matmul(L, L)
```

In [ ]:
```
# closed form solution you derived above
# x = np.matmul(np.linalg.inv(M + alpha * P), M) @ y
```

In [ ]:
```
# Try to experiment with different alpha values
# alpha =
for alpha in [0.01, 0.1, 1, 10]:
  x = np.matmul(np.linalg.inv(M + alpha * P), M) @ y
  reconstructed_image = np.reshape(x, (height_img, width_img))

  fig, (ax1, ax2) = plt.subplots(1, 2)
  ax1.imshow(corrupted_image, cmap=plt.get_cmap('gray'))
  ax1.set_title("Corrupted Image")
  ax2.imshow(reconstructed_image, cmap=plt.get_cmap('gray'))
  ax2.set_title("Reconstructed Image")

  plt.show()
```







4. Discuss the influence of the smoothing parameter α in the optimization problem above. What happens for very large and very low values of α? Finally, discuss the degree of our polynomial function $\mathbf{P}$. What happens if we would choose a large degree?

- The influence of the smoothing parameter α
  - α is very large
    - The regularization term has much weight, so the optimization focuses on the smoothness of the reconstructed image
    - The reconstructed images are blurry and the reconstructed parts are continuous with their neighbors
  - α is very small
    - The regularization term is negligble, so the optimization emphasizes on matching with the observed data
    - The reconstructed parts of the image are more discontinuous and do not have smooth transitions with their neighboring pixels
- The influence of the degree of the polynomial function $\mathbf{P}$
  - The degree of $\mathbf{L}$ represents the steps a pixel needs to take to get to another pixel
  - If the degree is large, $\mathbf{P}$ can connect pixels that have a long distance with each other, enabling the smoothing across a large area of the picture. This might make the image blurrier.

Problem 2: Protein classification using GNNs (20 points)

In this problem, we focus on a **graph classification** problem using graph neural networks. Specifically, we employ GNNs to generate graph embeddings (a vector representation for each graph), which are then passed through a linear layer for classification. A common graph classification task is molecular property prediction, where molecules are represented as graphs derived from their chemical structures, and the goal is to predict their properties.

Q1: What are some advantages of using GNN and graph representations for molecular property prediction compared to using a fully connected neural networks with tabular data? (5 points)

GNN keeps the structures and relationships between nodes while a fully connected neural network with tabular data doesn't. In the molecules case, GNN is better since it can keep the structure of molecules in training.

Here, we'll use the PROTEINS datatset from TUDataset and our task is to **train a model to predict if the protein is an enzyme or not**.

In [ ]:
```
import os
import torch
```

```
os.environ['TORCH'] = torch.__version__
print(torch.__version__)

!pip install -q torch-scatter -f https://data.pyg.org/whl/torch-${TORCH}.html
!pip install -q torch-sparse -f https://data.pyg.org/whl/torch-${TORCH}.html
!pip install -q git+https://github.com/pyg-team/pytorch_geometric.git

import torch
from torch_geometric.datasets import TUDataset
from torch_geometric.utils import to_networkx
import networkx as nx
import matplotlib.pyplot as plt
```

```
2.8.0+cu126
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.9/10.9 MB 122.4 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.2/5.2 MB 73.3 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for torch-geometric (pyproject.toml) ... done
```

In [ ]:
```
dataset = TUDataset(root='data/TUDataset', name='PROTEINS')

# general graph information about the whole datasets
print()
print(f'Dataset: {dataset}:')
print('====================')
print(f'Number of graphs: {len(dataset)}')
print(f'Number of features: {dataset.num_features}')
print(f'Number of classes: {dataset.num_classes}')
```

```
Downloading https://www.chrsmrrs.com/graphkerneldatasets/PROTEINS.zip
Processing...
Dataset: PROTEINS(1113):
====================
Number of graphs: 1113
Number of features: 3
Number of classes: 2
Done!
```

As shown above, the dataset contains 1,113 proteins with binary labels, where 1 denotes an enzyme and 0 denotes a non-enzyme. We next examine and visualize the first graph in the dataset to gain a better understanding of its structure.

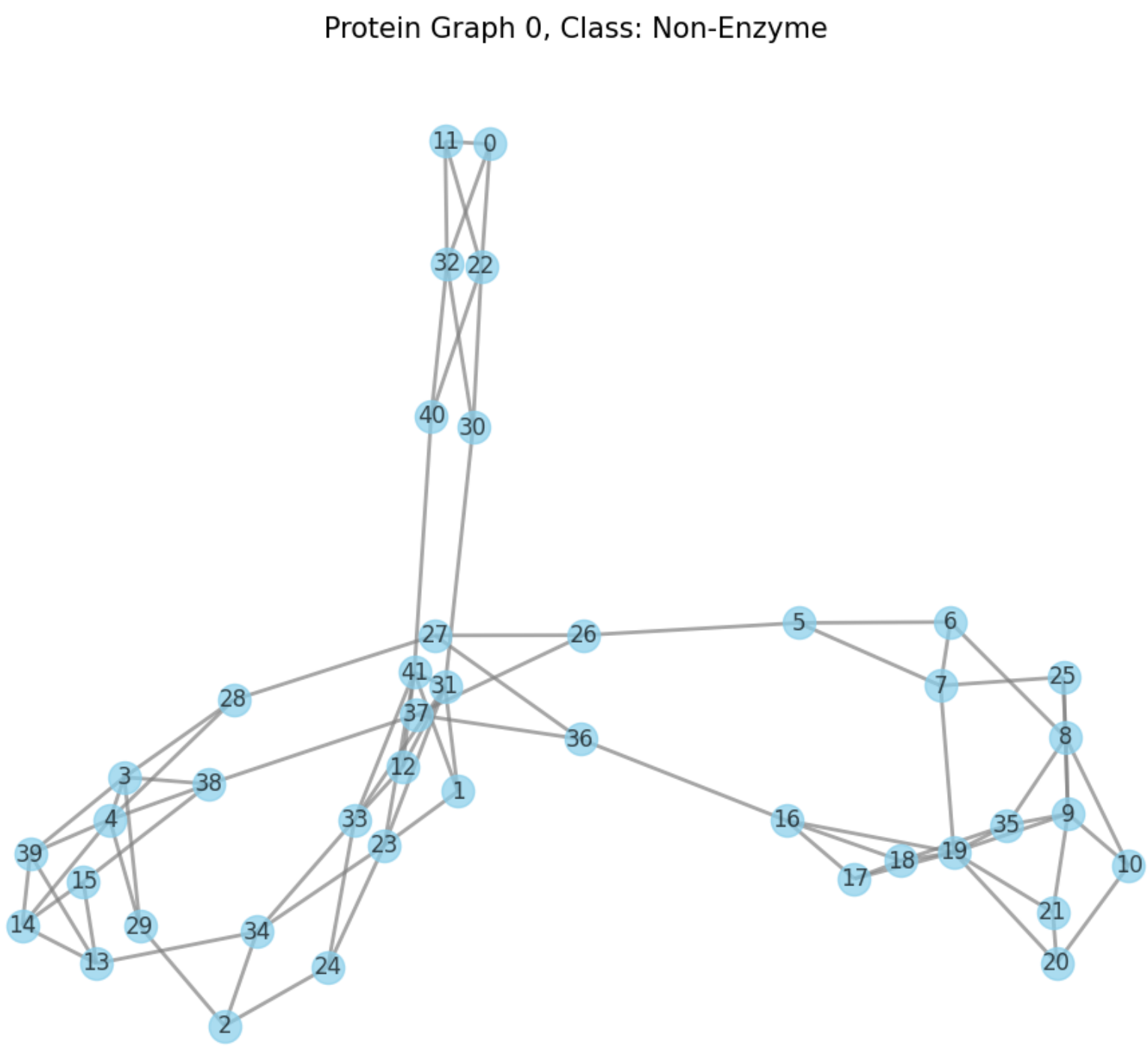
In [ ]:
```
# taking a closer look at the first graph
data = dataset[0]

print()
print(data)
print('=============================================================')

# Gather some statistics about the first graph.
print(f'Number of nodes: {data.num_nodes}')
print(f'Number of edges: {data.num_edges}')
print(f'Average node degree: {data.num_edges / data.num_nodes:.2f}')
print(f'Has isolated nodes: {data.has_isolated_nodes()}')
print(f'Has self-loops: {data.has_self_loops()}')
print(f'Is undirected: {data.is_undirected()}')
```

```
Data(edge_index=[2, 162], x=[42, 3], y=[1])
=============================================================
Number of nodes: 42
Number of edges: 162
Average node degree: 3.86
Has isolated nodes: False
Has self-loops: False
Is undirected: True
```

The first graph contains 42 nodes each with 3 features and has 162 edges. A visualization of this graph is provided below.

In [ ]:
```
# Convert to NetworkX using the built-in utility
G = to_networkx(data, to_undirected=True)

# Visualization
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G, seed=42)

nx.draw(G, pos,
        node_color='skyblue',
        node_size=300,
        with_labels=True,
        width=2,
        edge_color='gray',
```

```
        alpha=0.7)

plt.title(f"Protein Graph 0, Class: {"Enzyme" if data.y ==1 else "Non-Enzyme" }", fontsize=15)
plt.show()
```



In the following, similar to all machine learning model training pipeline, we split the dataset, batch them, and start our model building process.

In [ ]:
```
torch.manual_seed(12345)
dataset = dataset.shuffle()

train_dataset = dataset[:800]
test_dataset = dataset[800:]

print(f'Number of training graphs: {len(train_dataset)}')
print(f'Number of test graphs: {len(test_dataset)}')

from torch_geometric.loader import DataLoader

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

```
Number of training graphs: 800
Number of test graphs: 313
```

A GNN model for graph classification tasks usually come with the following structure:

1. Embed each node by performing multiple rounds of message passing using layers such as Graph Convolutional Layer (more on this later)
2. Aggregate node embeddings into a unified graph embedding (through pooling methods, such as taking the mean, max, sum)
3. Train a final classifier on the graph embedding

A bit more on Graph Convolutional Layers (GCN): The mathematical operation GCN performs are described in detail here.

In simple terms, during a GCN operation, each node's new representation is obtained by aggregating (typically through a

weighted average) the current representations of itself and its neighbors, followed by a linear transformation. As in fully connected neural networks, an activation function is usually applied after each GCN layer.

Its node-wise formulation is given by:

Weighted Average

$$\mathbf{x}_i' = \mathbf{\Theta}^\top \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{e_{j,i}}{\sqrt{\hat{d}_j \hat{d}_i}} \mathbf{x}_j$$

with $\hat{d}_i = 1 + \sum_{j \in \mathcal{N}(i)} e_{j,i}$, where $e_{j,i}$ denotes the edge weight from source node `j` to target node `i` (default: `1.0`)

Q2: Fill in the necessary code for GCN forward passing based on the description above and the example for the first layer (5 points)

In [ ]:
```
from torch.nn import Linear
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
from torch_geometric.nn import global_mean_pool

class GCN(torch.nn.Module):
    def __init__(self, hidden_channels):
        super(GCN, self).__init__()
        torch.manual_seed(12345)
        self.conv1 = GCNConv(dataset.num_node_features, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, hidden_channels)
        self.conv3 = GCNConv(hidden_channels, hidden_channels)
        self.lin = Linear(hidden_channels, dataset.num_classes)

    def forward(self, x, edge_index, batch):
        # 1. Obtain node embeddings
        x = self.conv1(x, edge_index) # first GCN layer operation
        x = x.relu()                  # activation function
        ##############################
        # write your code here: how do we do GCN operation for the second and third layer
        # approximately 4 lines of code (aggregation + activation)
        x = self.conv2(x, edge_index)
        x = x.relu()
        x = self.conv3(x, edge_index)
        x = x.relu()

        ##############################

        # 2. Pooling layer
        x = global_mean_pool(x, batch)  # turn node embedding into graph embedding by taking the mean of al

        # 3. Apply a final classifier
        x = F.dropout(x, p=0.5, training=self.training)
        x = self.lin(x)

        return x

model = GCN(hidden_channels=64)
print(model)
```

```
GCN(
  (conv1): GCNConv(3, 64)
  (conv2): GCNConv(64, 64)
  (conv3): GCNConv(64, 64)
  (lin): Linear(in_features=64, out_features=2, bias=True)
)
```

Q3: After generating graph embeddings, graph classification follows the standard classification pipeline. For our setup, what loss function is appropriate? Briefly explain your choice. Fill in your chosen loss function in the second line of the following code block. Feel free to check the loss function provided in torch here. (5 Points)

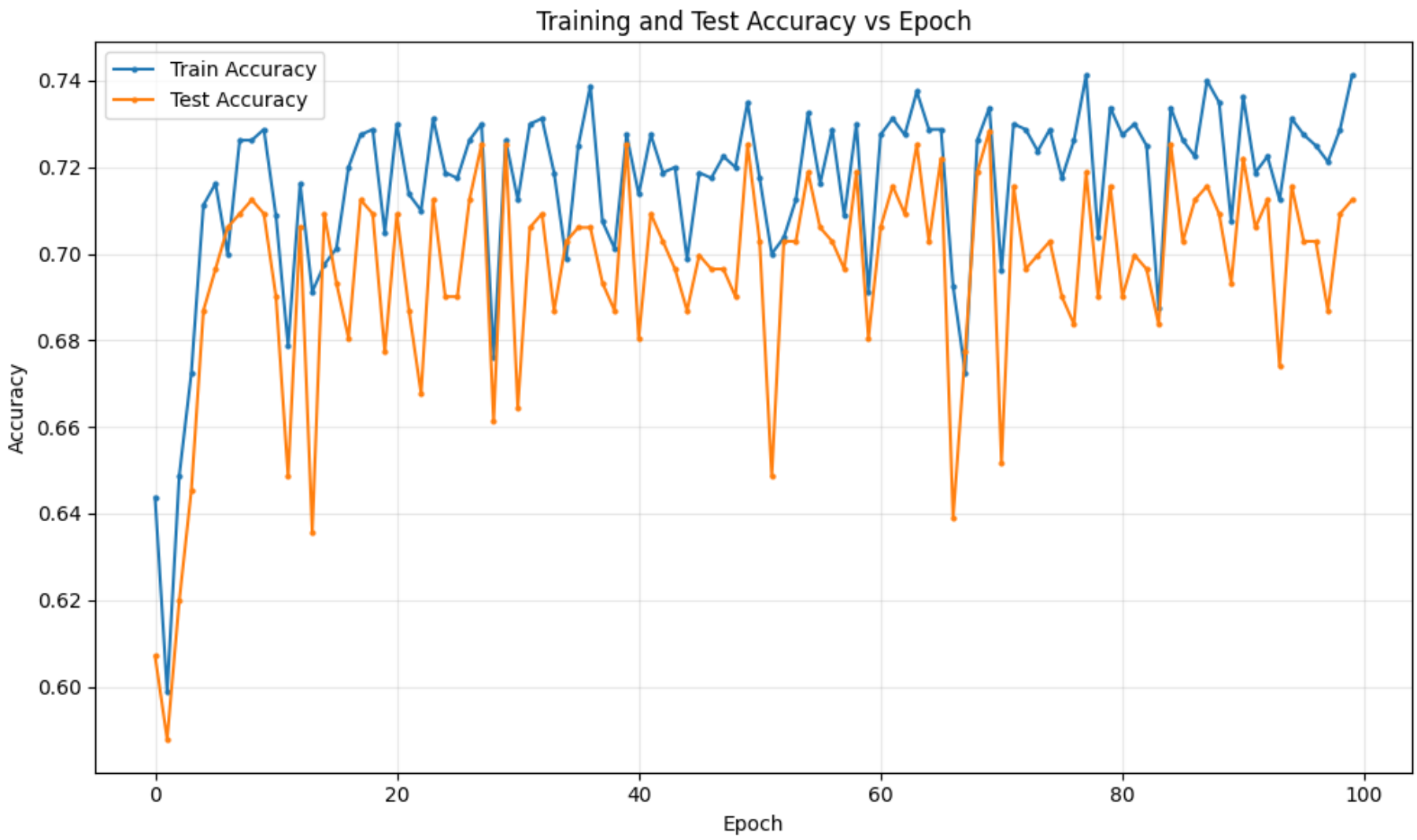
Since our task is a binary classification task, I choose `torch.nn.CrossEntropyLoss` here, treating a binary classification as a special case for a multi-classification.

In [ ]:
```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
 ## fill in your loss function here
criterion = torch.nn.CrossEntropyLoss()

def train():
    model.train()
```

```
    for data in train_loader:  # Iterate in batches over the training dataset.
         out = model(data.x, data.edge_index, data.batch)  # Perform a single forward pass.
         loss = criterion(out, data.y)  # Compute the loss.
         loss.backward()  # Derive gradients.
         optimizer.step()  # Update parameters based on gradients.
         optimizer.zero_grad()  # Clear gradients.

def test(loader):
     model.eval()

     correct = 0
     for data in loader:  # Iterate in batches over the training/test dataset.
         out = model(data.x, data.edge_index, data.batch)
         pred = out.argmax(dim=1)  # Use the class with highest probability.
         correct += int((pred == data.y).sum())  # Check against ground-truth labels.
     return correct / len(loader.dataset)  # Derive ratio of correct predictions.

train_accs = []
test_accs = []

for epoch in range(1, 101):
    train()
    train_acc = test(train_loader)
    test_acc = test(test_loader)
    train_accs.append(train_acc)
    test_accs.append(test_acc)
    print(f'Epoch: {epoch:03d}, Train Acc: {train_acc:.4f}, Test Acc: {test_acc:.4f}')

import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.plot(range(len(train_accs)), train_accs, label='Train Accuracy', marker='o', markersize=2)
plt.plot(range(len(train_accs)), test_accs, label='Test Accuracy', marker='o', markersize=2)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Test Accuracy vs Epoch')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Epoch: 001, Train Acc: 0.6438, Test Acc: 0.6070
Epoch: 002, Train Acc: 0.5988, Test Acc: 0.5879
Epoch: 003, Train Acc: 0.6488, Test Acc: 0.6198
Epoch: 004, Train Acc: 0.6725, Test Acc: 0.6454
Epoch: 005, Train Acc: 0.7113, Test Acc: 0.6869
Epoch: 006, Train Acc: 0.7163, Test Acc: 0.6965
Epoch: 007, Train Acc: 0.7000, Test Acc: 0.7061
Epoch: 008, Train Acc: 0.7262, Test Acc: 0.7093
Epoch: 009, Train Acc: 0.7262, Test Acc: 0.7125
Epoch: 010, Train Acc: 0.7288, Test Acc: 0.7093
Epoch: 011, Train Acc: 0.7087, Test Acc: 0.6901
Epoch: 012, Train Acc: 0.6787, Test Acc: 0.6486
Epoch: 013, Train Acc: 0.7163, Test Acc: 0.7061
Epoch: 014, Train Acc: 0.6913, Test Acc: 0.6358
Epoch: 015, Train Acc: 0.6975, Test Acc: 0.7093
Epoch: 016, Train Acc: 0.7013, Test Acc: 0.6933
Epoch: 017, Train Acc: 0.7200, Test Acc: 0.6805
Epoch: 018, Train Acc: 0.7275, Test Acc: 0.7125
Epoch: 019, Train Acc: 0.7288, Test Acc: 0.7093
Epoch: 020, Train Acc: 0.7050, Test Acc: 0.6773
Epoch: 021, Train Acc: 0.7300, Test Acc: 0.7093
Epoch: 022, Train Acc: 0.7137, Test Acc: 0.6869
Epoch: 023, Train Acc: 0.7100, Test Acc: 0.6677
Epoch: 024, Train Acc: 0.7312, Test Acc: 0.7125
Epoch: 025, Train Acc: 0.7188, Test Acc: 0.6901
Epoch: 026, Train Acc: 0.7175, Test Acc: 0.6901
Epoch: 027, Train Acc: 0.7262, Test Acc: 0.7125
Epoch: 028, Train Acc: 0.7300, Test Acc: 0.7252
Epoch: 029, Train Acc: 0.6763, Test Acc: 0.6613
Epoch: 030, Train Acc: 0.7262, Test Acc: 0.7252
Epoch: 031, Train Acc: 0.7125, Test Acc: 0.6645
Epoch: 032, Train Acc: 0.7300, Test Acc: 0.7061
Epoch: 033, Train Acc: 0.7312, Test Acc: 0.7093
Epoch: 034, Train Acc: 0.7188, Test Acc: 0.6869
Epoch: 035, Train Acc: 0.6987, Test Acc: 0.7029
Epoch: 036, Train Acc: 0.7250, Test Acc: 0.7061
Epoch: 037, Train Acc: 0.7388, Test Acc: 0.7061
Epoch: 038, Train Acc: 0.7075, Test Acc: 0.6933
Epoch: 039, Train Acc: 0.7013, Test Acc: 0.6869
Epoch: 040, Train Acc: 0.7275, Test Acc: 0.7252
Epoch: 041, Train Acc: 0.7137, Test Acc: 0.6805
```

```
Epoch: 042, Train Acc: 0.7275, Test Acc: 0.7093
Epoch: 043, Train Acc: 0.7188, Test Acc: 0.7029
Epoch: 044, Train Acc: 0.7200, Test Acc: 0.6965
Epoch: 045, Train Acc: 0.6987, Test Acc: 0.6869
Epoch: 046, Train Acc: 0.7188, Test Acc: 0.6997
Epoch: 047, Train Acc: 0.7175, Test Acc: 0.6965
Epoch: 048, Train Acc: 0.7225, Test Acc: 0.6965
Epoch: 049, Train Acc: 0.7200, Test Acc: 0.6901
Epoch: 050, Train Acc: 0.7350, Test Acc: 0.7252
Epoch: 051, Train Acc: 0.7175, Test Acc: 0.7029
Epoch: 052, Train Acc: 0.7000, Test Acc: 0.6486
Epoch: 053, Train Acc: 0.7037, Test Acc: 0.7029
Epoch: 054, Train Acc: 0.7125, Test Acc: 0.7029
Epoch: 055, Train Acc: 0.7325, Test Acc: 0.7188
Epoch: 056, Train Acc: 0.7163, Test Acc: 0.7061
Epoch: 057, Train Acc: 0.7288, Test Acc: 0.7029
Epoch: 058, Train Acc: 0.7087, Test Acc: 0.6965
Epoch: 059, Train Acc: 0.7300, Test Acc: 0.7188
Epoch: 060, Train Acc: 0.6913, Test Acc: 0.6805
Epoch: 061, Train Acc: 0.7275, Test Acc: 0.7061
Epoch: 062, Train Acc: 0.7312, Test Acc: 0.7157
Epoch: 063, Train Acc: 0.7275, Test Acc: 0.7093
Epoch: 064, Train Acc: 0.7375, Test Acc: 0.7252
Epoch: 065, Train Acc: 0.7288, Test Acc: 0.7029
Epoch: 066, Train Acc: 0.7288, Test Acc: 0.7220
Epoch: 067, Train Acc: 0.6925, Test Acc: 0.6390
Epoch: 068, Train Acc: 0.6725, Test Acc: 0.6773
Epoch: 069, Train Acc: 0.7262, Test Acc: 0.7188
Epoch: 070, Train Acc: 0.7338, Test Acc: 0.7284
Epoch: 071, Train Acc: 0.6963, Test Acc: 0.6518
Epoch: 072, Train Acc: 0.7300, Test Acc: 0.7157
Epoch: 073, Train Acc: 0.7288, Test Acc: 0.6965
Epoch: 074, Train Acc: 0.7238, Test Acc: 0.6997
Epoch: 075, Train Acc: 0.7288, Test Acc: 0.7029
Epoch: 076, Train Acc: 0.7175, Test Acc: 0.6901
Epoch: 077, Train Acc: 0.7262, Test Acc: 0.6837
Epoch: 078, Train Acc: 0.7412, Test Acc: 0.7188
Epoch: 079, Train Acc: 0.7037, Test Acc: 0.6901
Epoch: 080, Train Acc: 0.7338, Test Acc: 0.7157
Epoch: 081, Train Acc: 0.7275, Test Acc: 0.6901
Epoch: 082, Train Acc: 0.7300, Test Acc: 0.6997
Epoch: 083, Train Acc: 0.7250, Test Acc: 0.6965
Epoch: 084, Train Acc: 0.6875, Test Acc: 0.6837
Epoch: 085, Train Acc: 0.7338, Test Acc: 0.7252
Epoch: 086, Train Acc: 0.7262, Test Acc: 0.7029
Epoch: 087, Train Acc: 0.7225, Test Acc: 0.7125
Epoch: 088, Train Acc: 0.7400, Test Acc: 0.7157
Epoch: 089, Train Acc: 0.7350, Test Acc: 0.7093
Epoch: 090, Train Acc: 0.7075, Test Acc: 0.6933
Epoch: 091, Train Acc: 0.7362, Test Acc: 0.7220
Epoch: 092, Train Acc: 0.7188, Test Acc: 0.7061
Epoch: 093, Train Acc: 0.7225, Test Acc: 0.7125
Epoch: 094, Train Acc: 0.7125, Test Acc: 0.6741
Epoch: 095, Train Acc: 0.7312, Test Acc: 0.7157
Epoch: 096, Train Acc: 0.7275, Test Acc: 0.7029
Epoch: 097, Train Acc: 0.7250, Test Acc: 0.7029
Epoch: 098, Train Acc: 0.7212, Test Acc: 0.6869
Epoch: 099, Train Acc: 0.7288, Test Acc: 0.7093
Epoch: 100, Train Acc: 0.7412, Test Acc: 0.7125
```



Q4: In GNNs, adding more layers doesn't always improve performance and can sometimes dramatically decrease performance on both training and test sets. You might suspect overfitting, but overfitting wouldn't degrade training performance. This phenomenon is called oversmoothing. Based on the name and the "aggregation + activation" operation performed by each layer, what might cause this issue? (Hint: As we stack more layers, we aggregate information from increasingly distant neighbors, and...) (5 Points)

- Each layer performs the aggregation and activation, which means the layer is combining the feature of the node and its neighbors
- If we add the layers, the node can have access to a more distant neighbor. If the GNN is too deep, every node might be able to access the whole image and combine the features of all the nodes while ignores the local features, which mgiht cause the oversmoothing. That's why both the training and test performance decrease.

Problem 3: Positive reinforcement (10 points)

As discussed in class, reinforcement learning using policy gradient methods is based on maximizing the expected total reward

$$J(\theta) = \mathbb{E}_\theta[R(\tau)],$$

where the expectation is over the probability distribution over sequences τ through a choice of actions using the policy. This can be rewritten as

$$\nabla_\theta J(\theta) = \mathbb{E}_\theta \left[R(\tau) \nabla_\theta \log p(\tau \mid \theta) \right].$$

Approximating this gradient involves computing $\nabla_\theta \log \pi_\theta(a \mid s)$ where π_θ is the policy.

3.1 Continuous action space with Gaussian policy

Suppose that the action space is continuous and $\pi_\theta(a \mid s)$ is a normal density with mean $\mu_\theta(s)$ and variance $\sigma^2_\theta(s)$, two outputs of a neural network with input s and parameters θ.

Suppose the outputs of the neural network are given by

$$\mu_\theta(s) = \beta_1^T h(s)$$
$$\sigma^2_\theta(s) = \exp(\beta_2^T h(s))$$

where $h(s)$ is the vector of neurons in the last layer, immediately before the outputs. Derive explicit expressions for $\nabla_{\beta_1} \log \pi_\theta(a \mid s)$ and $\nabla_{\beta_2} \log \pi_\theta(a \mid s)$.

Explain how these gradients and other gradient terms in $\nabla_\theta \log \pi_\theta(a \mid s)$ are used to estimate the policy.

3.1

$\pi_\theta(a|s)$ is the density of $N(\mu_\theta(s), \sigma_\theta^2(s)) \Rightarrow \pi_\theta(a|s) = \frac{1}{\sqrt{2\pi}\sigma_\theta(s)} e^{-\frac{(a-\mu_\theta(s))^2}{2\sigma_\theta^2(s)}}$

$\Rightarrow \log \pi_\theta(a|s) = -\log(\sqrt{2\pi}\sigma_\theta(s)) - \frac{(a-\mu_\theta(s))^2}{2\sigma_\theta^2(s)} = -\frac{1}{2}\log(2\pi\sigma_\theta^2(s)) - \frac{(a-\mu_\theta(s))^2}{2\sigma_\theta^2(s)}$

Since $\mu_\theta(s) = \beta_1^T h(s)$, $\sigma_\theta^2(s) = \exp(\beta_2^T h(s))$. only $\mu_\theta(s)$ depends on β_1. only $\sigma_\theta(s)$ depends on β_2.

$\frac{\partial}{\partial \beta_1}\mu_\theta(s) = \frac{\partial}{\partial \beta_1}\beta_1^T h(s) = h(s)$. $\frac{\partial}{\partial \beta_2}\sigma_\theta^2(s) = \frac{\partial}{\partial \beta_2}\exp(\beta_2^T h(s)) = \exp(\beta_2^T h(s))\frac{\partial}{\partial \beta_2}(\beta_2^T h(s)) = \sigma_\theta^2(s)\, h(s)$

$\nabla_{\beta_1}\log\pi_\theta(a|s) = \frac{\partial}{\partial \beta_1}\log\pi_\theta(a|s) = \frac{\partial}{\partial \beta_1}\left(-\frac{1}{2}\log(2\pi\sigma_\theta^2(s)) - \frac{(a-\mu_\theta(s))^2}{2\sigma_\theta^2(s)}\right) = \frac{1}{2\sigma_\theta^2(s)} \cdot 2(a-\mu_\theta(s)) \cdot \frac{\partial}{\partial \beta_1}\mu_\theta(s) = \frac{h(s)(a-\beta_1^T h(s))}{\exp(\beta_2^T h(s))}$

$\nabla_{\beta_2}\log\pi_\theta(a|s) = \frac{\partial}{\partial \beta_2}\left(-\frac{1}{2}\log(2\pi\sigma_\theta^2(s)) - \frac{(a-\mu_\theta(s))^2}{2\sigma_\theta^2(s)}\right) = -\frac{2\pi}{2\cdot 2\pi\sigma_\theta^2(s)}\frac{\partial}{\partial \beta_2}\sigma_\theta^2(s) - \frac{(a-\mu_\theta(s))^2}{2}\frac{-1}{(\sigma_\theta^2(s))^2}\frac{\partial}{\partial \beta_2}\sigma_\theta^2(s)$

$= \left(-\frac{1}{2\sigma_\theta^2(s)} + \frac{(a-\mu_\theta(s))^2}{2(\sigma_\theta^2(s))^2}\right)\sigma_\theta^2(s)\, h(s) = \frac{1}{2}\left(\frac{(a-\beta_1^T h(s))^2}{\exp(\beta_2^T h(s))} - 1\right) h(s)$

How they are used:

① sample N sequences $\{\tau^{(i)}\}$ by rolling out the current policy π_θ. each sequence consists of $(s_t^{(i)}, r_t^{(i)}, a_t^{(i)})$

② For each (action, state) pair $(a_t^{(i)}, s_t^{(i)})$, evaluate $\nabla_\theta \log\pi_\theta(a_t^{(i)}|s_t^{(i)})$ using the gradients $\nabla_{\beta_1}\log\pi_\theta(a_t^{(i)}|s_t^{(i)})$, $\nabla_{\beta_2}\log\pi_\theta(a_t^{(i)}|s_t^{(i)})$ and other gradients for parameters in the hidden layers.

③ $\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)})\nabla_\theta \log P(\tau^{(i)}|\theta) = \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)})\sum_{t=0}^{T}\nabla_\theta \log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

④ update θ: $\theta \leftarrow \theta + \eta\left(\frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)})\sum_{t=0}^{T}\nabla_\theta \log\pi_\theta(a_t^{(i)}|s_t^{(i)})\right)$

3.2 Discrete action space with Softmax policy

Suppose the action space is discrete with K possible actions, and the policy $\pi_\theta(a \mid s)$ is defined using a softmax function over preferences $u_\theta(s, a)$:

$$\pi_\theta(a \mid s) = \frac{\exp(u_\theta(s, a))}{\sum_a \exp(u_\theta(s, a))},$$

where $u_\theta(s, a) = \beta^T h(s, a)$, and $h(s, a)$ is a feature vector for state-action pair (s, a). Derive the expression for $\nabla_\beta \log \pi_\theta(a \mid s)$.

3.2

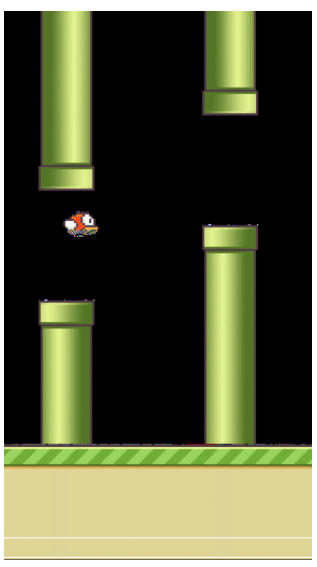
$\pi_\theta(a|s) = \frac{\exp(\mu_\theta(s,a))}{\sum_{a'}\exp(\mu_\theta(s,a'))}$ $\mu_\theta(s,a) = \beta^T h(s,a)$.

$\log\pi_\theta(a|s) = \mu_\theta(s,a) - \log\left(\sum_{a'}\exp(\mu_\theta(s,a'))\right) = \beta^T h(s,a) - \log\left(\sum_{a'}\exp(\beta^T h(s,a'))\right)$

$\nabla_\beta \log\pi_\theta(a|s) = \frac{\partial}{\partial\beta}\left(\beta^T h(s,a) - \log\left(\sum_{a'}\exp(\beta^T h(s,a'))\right)\right) = h(s,a) - \frac{\sum_{a'} h(s,a')\cdot\exp(\beta^T h(s,a'))}{\sum_{a'}\exp(\beta^T h(s,a'))} = h(s,a) - \sum_{a'} h(s,a')\cdot\pi_\theta(a'|s)$

Problem 4: Deep Q-Learning for Flappy Bird (25 points)

Deep Q-learning was proposed (and patented) by DeepMind and made a big splash when the same deep neural network architecture was shown to be able to surpass human performance on many different Atari games, playing directly from the pixels. In this problem, we will walk you through the implementation of deep Q-learning to learn to play the Flappy Bird game.



The implementation is based these references:

- DeepLearningFlappyBird
- Deep Q-Learning for Atari Breakout

We use the `pygame` package to visualize the interaction between the algorithm and the game environment. However, *pygame* is not well supported by Google Colab; we recommend you to run the code for this problem locally. A window will be popped up that displays the game as it progress in real-time (as for the Cartpole demo from class).

This problem is structured as follows:

- Load necessary packages
- Test the visualization of the game, to make sure everything's working
- Process the images to reduce the dimension
- Setup the game history buffer
- Implement the core Q-learning function
- Run the learning algorithm
- Interpret the results

Introduction

The Flappy Bird game is requires a few Python packages. Please install these *as soon as possible*, and notify us of any issues you experience so that we can help. The Python files can also be found on Canvas and in our GitHub repo at https://github.com/YData123/sds365-fa25/tree/main/assignments/assn4 .

In [1]:
```
# %pip install pygame
import numpy as np
import cv2
from collections import deque
import random
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, initializers
import wrapped_flappy_bird as flappy_bird

# In order to write the wrapped_flappy_bird utilities, you'll need to upload the following
# three files to /content on Colab: wrapped_flappy_bird.py, flappy_bird_utils.py, assets.zip.
# Then, unzip the file assets.zip -- you can do this by opening a terminal.
```

```
pygame 2.6.1 (SDL 2.28.4, Python 3.12.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
libpng warning: iCCP: known incorrect sRGB profile
```

The Flappy Bird environment

Interaction with the game environment is carried out through calls of the form

`(image, reward, terminal) = game.frame_step(action)`

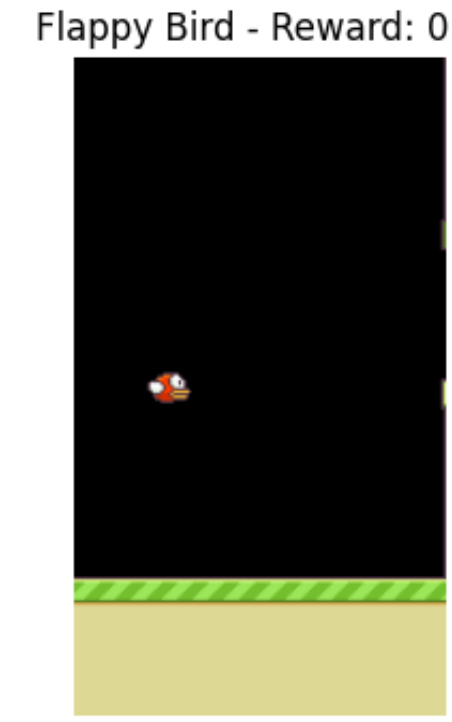
where the meaning of these variables is as follows:

- `action` : $\binom{1}{0}$ for doing nothing, $\binom{0}{1}$ for "flapping the bird's wings"
- `image` : the image for the next step of the game, of size $(288, 512, 3)$ with three RGB channels
- `reward` : the reward received for taking the action; -1 if an obstacle is hit, 0.1 otherwise.
- `terminal` : `True` if an obstacle is hit, otherwise `False`

Now let's take a look at the game interface. First, initiate the game:

In [2]:
```
num_actions = 2
```

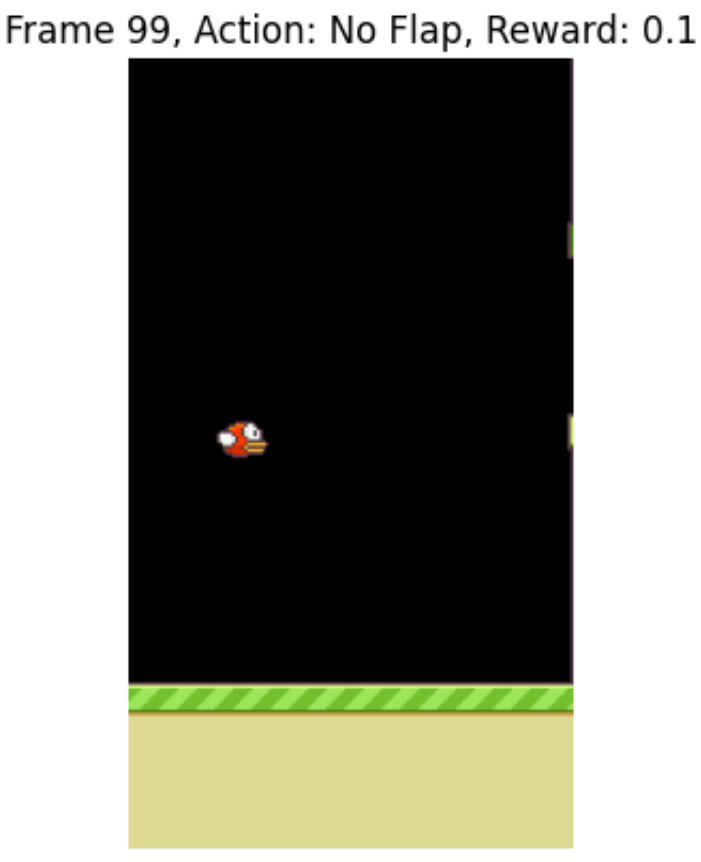
```
# initiate a game
game = flappy_bird.GameState()

# get the first state by doing nothing
do_nothing = np.zeros(num_actions)
do_nothing[0] = 1
image, reward, terminal = game.frame_step(do_nothing)

import matplotlib.pyplot as plt
%matplotlib inline
plt.figure(figsize=(8, 4))
plt.imshow(image.transpose([1, 0, 2]))
plt.axis('off')
plt.title(f'Flappy Bird - Reward: {reward}')
plt.show()

print('shape of image:', image.shape)
print('reward: ', reward)
print('terminal: ', terminal)
```




```
shape of image: (288, 512, 3)
reward:  0.1
terminal:  False
```

Let's take some random actions and see what happens:

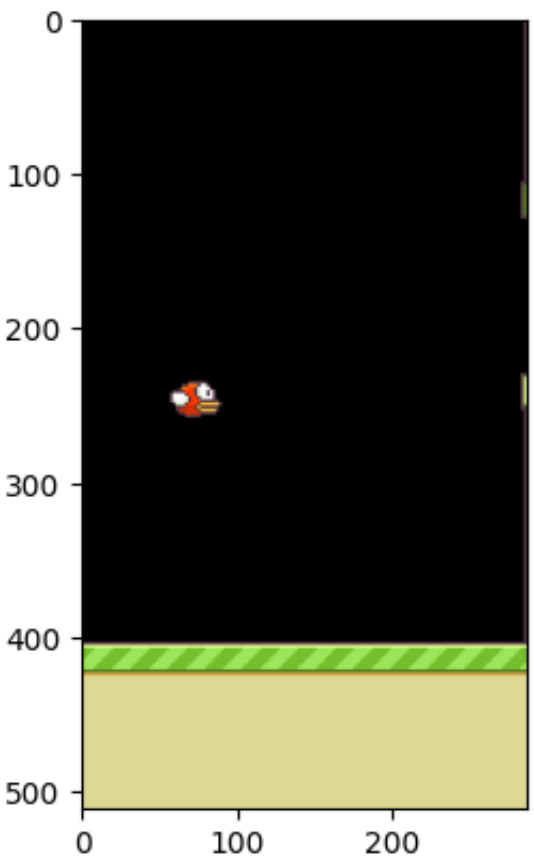
In [3]:
```
from IPython.display import clear_output
import time
for i in range(100):

    # choose a random action
    action = np.random.choice(num_actions)

    # create the corresponding one-hot vector
    action_vec = np.zeros(num_actions)
    action_vec[action] = 1

    # take the action and observe the reward and the next state
    image, reward, terminal = game.frame_step(action_vec)

    clear_output(wait=True)  # Clear previous frame
    time.sleep(.1)
    plt.imshow(image.transpose([1, 0, 2]))
    plt.axis('off')
    plt.title(f'Frame {i}, Action: {"Flap" if action == 0 else "No Flap"}, Reward: {reward}')
    plt.show()
```



Are you able to see Flappy moving across the window and crashing into things? Great! If you're having any issues, post to EdD and we'll do our best to help you out.

Here is how we can visualize a frame of the game as an image within a cell.

In [4]:
```
# show the image
import matplotlib.pyplot as plt
plt.imshow(image.transpose([1, 0, 2]))
plt.show()
plt.close()
```

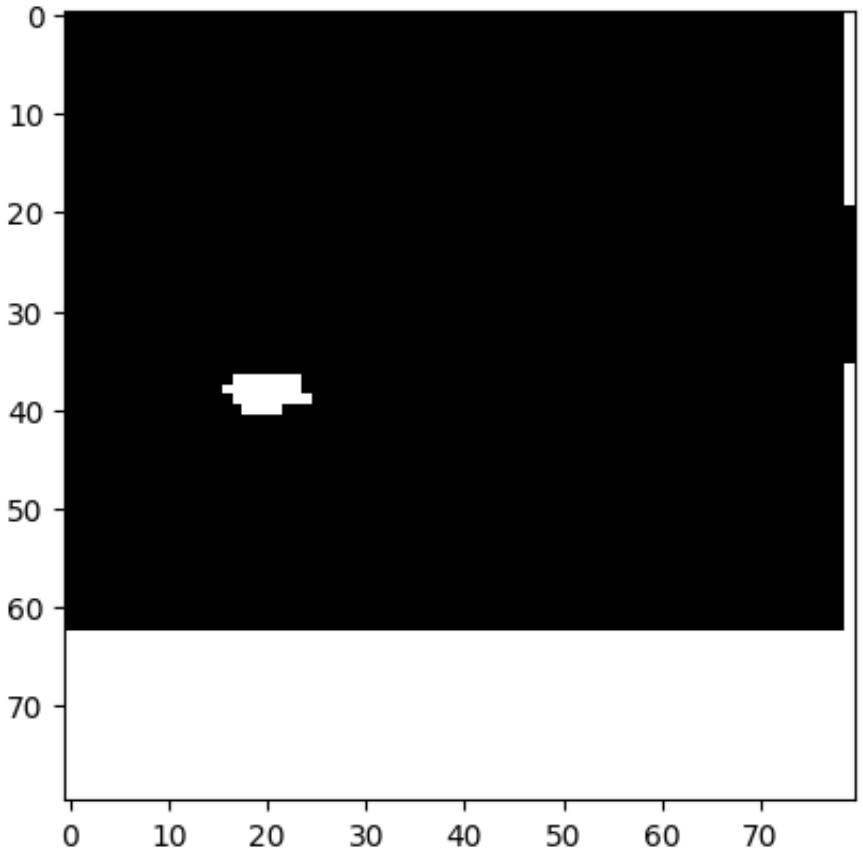


Preprocessing the images

Alright, next we need to prepocess the images by converting them to grayscale and resizing them to 80×80 pixels. This will help to reduce the computation, and aid learning. Besides, Flappy is "color blind." (Fun fact: The instructor of this course is also color vision deficient.)

In [5]:
```
def resize_gray(frame):
    frame = cv2.cvtColor(cv2.resize(frame, (80, 80)), cv2.COLOR_BGR2GRAY)
    ret, frame = cv2.threshold(frame, 1, 255, cv2.THRESH_BINARY)
    return np.reshape(frame, (80, 80, 1))

image_transformed = resize_gray(image)
print('Shape of the transformed image:', image.shape)

# show the transformed image
_ = plt.imshow(image_transformed.transpose((1, 0, 2)), cmap='gray')
```

```
Shape of the transformed image: (288, 512, 3)
```



This shows the preprocessed image for a single frame of the game. In our implementation of Deep Q-Learning, we encode the state by stacking four consecutive frames, resulting in a tensor of shape (80,80,4).

Then, given the `current_state`, and a raw image `image_raw` of size $288 \times 512 \times 3$, we convert the raw image to a $80 \times 80 \times 1$ grayscale image using the code in the previous cell. The , we remove the first frame of `current_state` and add the new frame, giving again a stack of images of size (80, 80, 4).

In [6]:
```python
def preprocess(image_raw, current_state=None):
    # resize and convert to grayscale
    image = resize_gray(image_raw)
    # stack the frames
    if current_state is None:
        state = np.concatenate((image, image, image, image), axis=2)
    else:
        state = np.concatenate((image, current_state[:, :, :3]), axis=2)
    return state
```

4.1 Explain the game state

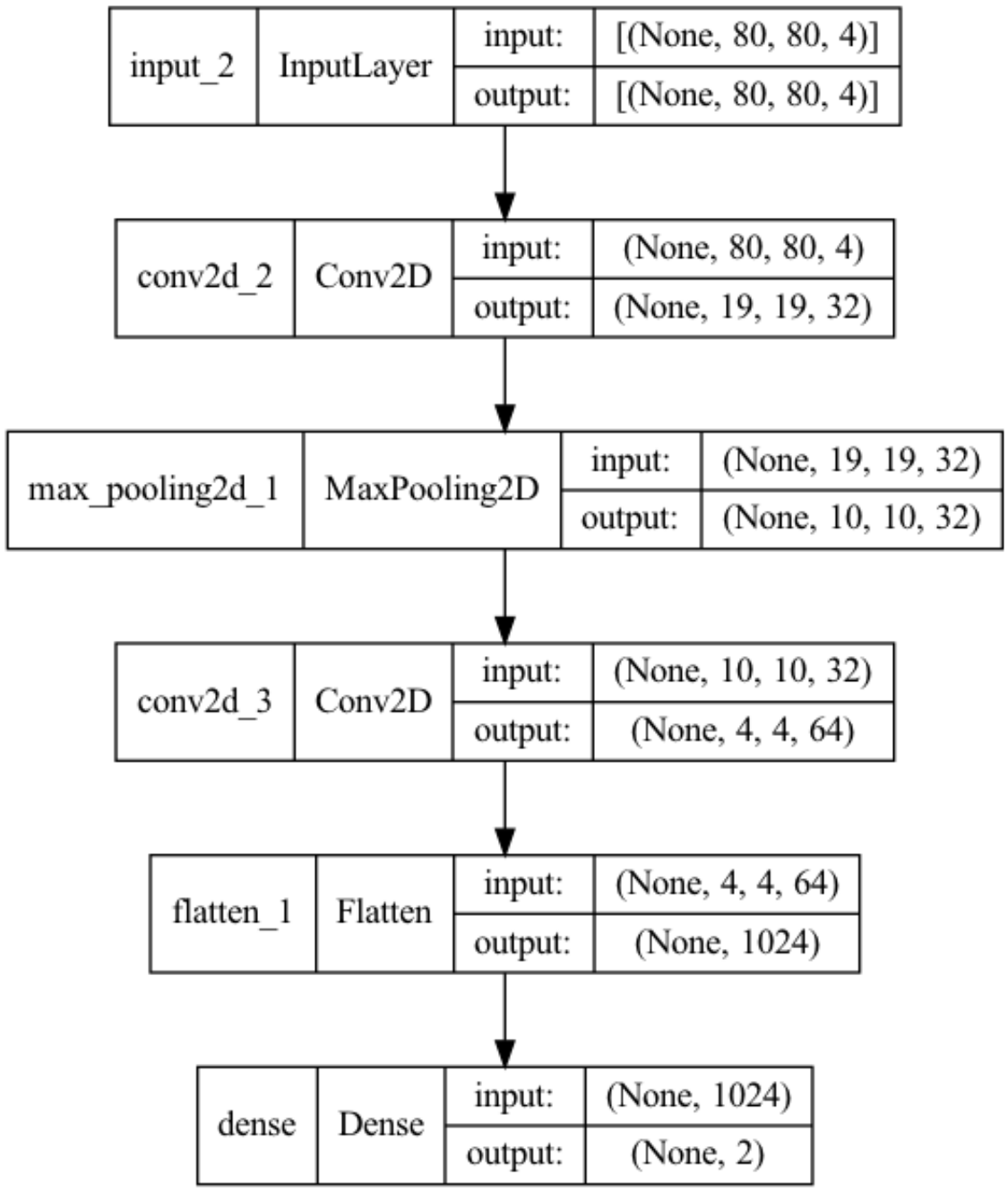
Why is the state chosen to be a stack of four consecutive frames rather than a single frame? Give an intuitive explanation.

The state of a stack of 4 consecutive frames can contain information not only about the bird's position, but also the bird's motion, which allows the model to do dynamic decision making.

Constructing the neural network

Now we are ready to construct the neural network for approximating the Q function. Recall that, given input s which is of size $80 \times 80 \times 4$ due to the previous preprocessing, the output of the network should be of size 2, corresponding to the values of $Q(s, a_1)$ and $Q(s, a_2)$ respectively.

Here is the summary of the model we'd like to build:



4.2 Initialize the network

Complete the code in the next cell so that your model architecture matches that in the above picture. Here we specify the initialization of the weights by using `keras.initializers`. Note that we haven't talked about the `strides` argument for CNNs; you can read about stride here: https://machinelearningmastery.com/padding-and-stride-for-convolutional-neural-networks/. It's not important to understand this in detail, you just need to choose the number and sizes of the filters to get the shapes to match the specification.

In [7]:
```
from tensorflow.keras import initializers
def create_q_model():
    state = layers.Input(shape=(80, 80, 4,))

    layer1 = layers.Conv2D(filters=32, kernel_size=8, strides=4, activation="relu",
                           kernel_initializer=initializers.TruncatedNormal(mean=0., stddev=0.01),
                           bias_initializer=initializers.Constant(0.01))(state)
    layer2 = layers.MaxPool2D(2, strides=2, padding="same")(layer1)
    layer3 = layers.Conv2D(filters=64, kernel_size=3, strides=2, activation="relu",
                           kernel_initializer=initializers.TruncatedNormal(mean=0., stddev=0.01),
                           bias_initializer=initializers.Constant(0.01))(layer2)
    layer4 = layers.Flatten()(layer3)
    q_value = layers.Dense(units=2, activation="linear",
                           kernel_initializer=initializers.TruncatedNormal(mean=0., stddev=0.01),
                           bias_initializer=initializers.Constant(0.01))(layer4)

    return keras.Model(inputs=state, outputs=q_value)
```

Plot the model summary to make sure that the network is the same as expected.

In [8]:
```
model = create_q_model()
print(model.summary())
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 80, 80, 4)	0
conv2d (Conv2D)	(None, 19, 19, 32)	8,224
max_pooling2d (MaxPooling2D)	(None, 10, 10, 32)	0
conv2d_1 (Conv2D)	(None, 4, 4, 64)	18,496
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 2)	2,050

Total params: 28,770 (112.38 KB)
Trainable params: 28,770 (112.38 KB)
Non-trainable params: 0 (0.00 B)
None

Deep Q-learning

We're now ready to implement the Q-learning algorithm. There are some subtle details in the implementation that you need to sort out. First, recall that the update rule for Q learning is

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r(s,a) + \gamma \cdot \max_{a'} Q(\text{next}(s,a), a') - Q(s,a))$$

where γ is the discount factor and α can be viewed as the step size or learning rate for gradient ascent.

We'll set these as follows:

In [9]:
```
gamma = 0.99            # decay rate of past observations
step_size = 1e-4        # step size
```

Estimation with experience replay

At the beginning of training, we spend 10,000 steps taking random actions, as a means of observing the environment.

We build a replay memory of length 10,000 steps, and every time we update the weights of the network, we sample a batch of size 32 and perform a Q-learning update on this batch.

After we have collected 10,000 steps of new data, we discard the old data, and replace it with the new "experiences."

In [10]:
```
observe = 10000             # timesteps to observe before training
replay_memory = 10000       # number of previous transitions to remember
batch_size = 32             # size of each batch
```

4.3 Justify the data collection

Why does it make sense to maintain the replay memory of a fixed size instead of including all of the historical data?

- As the training process forwards, the old experiences become less relevant with poor quality. To prevent the outdated experiences from decreasing the quality of future training, they should be erased as time goes by.
- Replay buffers can grow extremely large if all the historical data are included. To avoid insufficient memory space, the old experiences should be erased.

Exploration vs exploitation

When performing Q-learning, we face the tradeoff between exploration and exploitation. To encourage exploration, a simple strategy is to take a random action at each step with certain probability.

More precisely, for each time step t and state s_t, with probability ϵ, the algorithm takes a random action (wing flap or do nothing), and with probability $1 - \epsilon$ the algorithm takes a greedy action according to $a_t = \arg\max_a Q_\theta(s_t, a)$. Here θ refers to the parameters of our CNN.

In [11]:
```
# value of epsilon
epsilon = 0.05
```

4.4 Complete the Q-learning algorithm

Next you will need to complete the Q-learning algorithm by filling in the missing code in the following function. The missing parts include

- Taking a greedy action
- Given a batch of samples $\{(s_t, a_t, r_t, s_{t+1}, \text{terminal}_t)\}_{t \in B}$, computing the corresponding $Q_\theta(s_t, a_t)$.
- Given a batch of samples $\{(s_t, a_t, r_t, s_{t+1}, \text{terminal}_t)\}_{t \in B}$, computing the corresponding updated Q-values

$$\hat{y}(s_t, a_t) = \begin{cases} r_t + \gamma \max_a Q_\theta(s_{t+1}, a), & \text{if terminal}_t = 0, \\ r_t, & \text{otherwise.} \end{cases}$$

Then, the mean squared error loss for the batch is

$$\frac{1}{|B|} \sum_{t \in B} (\hat{y}(s_t, a_t) - Q_\theta(s_t, a_t))^2.$$

In [12]:
```
def dql_flappy_bird(model, optimizer, loss_function):

    # initiate a game
    game = flappy_bird.GameState()

    # store the previous state, action and transitions
    history_data = deque()

    # get the first observation by doing nothing and preprocess the image
    do_nothing = np.zeros(num_actions)
    do_nothing[0] = 1
    image, reward, terminal = game.frame_step(do_nothing)

    # preprocess to get the state
    current_state = preprocess(image_raw=image)

    # training
    t = 0

    while t < 50000:
        if epsilon > np.random.rand(1)[0]:
            # random action
            action = np.random.choice(num_actions)
        else:
            # compute the Q function
            current_state_tensor = tf.convert_to_tensor(current_state)
            current_state_tensor = tf.expand_dims(current_state_tensor, 0)
            q_value = model(current_state_tensor, training=False)

            # greedy action
            #-----MISSING-----#
            action = np.argmax(q_value[0])
            #-----------------#

        # take the action and observe the reward and the next state
        action_vec = np.zeros([num_actions])
        action_vec[action] = 1
        image_raw, reward, terminal = game.frame_step(action_vec)
        next_state = preprocess(current_state=current_state,
                                image_raw=image_raw)

        # store the observation
        history_data.append((current_state, action, reward, next_state,
                             terminal))
        if len(history_data) > replay_memory:
            history_data.popleft()  # discard old data

        # train if done observing
        if t > observe:

            # sample a batch
            batch = random.sample(history_data, batch_size)
            state_sample = np.array([d[0] for d in batch])
            action_sample = np.array([d[1] for d in batch])
            reward_sample = np.array([d[2] for d in batch])
            state_next_sample = np.array([d[3] for d in batch])
            terminal_sample = np.array([d[4] for d in batch])

            # compute the updated Q-values for the samples
            #-----MISSING-----#
            future_rewards = model(state_next_sample, training=False, verbose=0)
            max_future_q_values = tf.reduce_max(future_rewards, axis=1)
            updated_q_value = reward_sample + gamma * max_future_q_values * (1 - terminal_sample)
            #-----------------#
```

```
        # train the model on the states and updated Q-values
        with tf.GradientTape() as tape:
            # compute the current Q-values for the samples
            #-----MISSING-----#
            current_q_values = model(state_sample, training=True, verbose=0)
            action_sample_vec = np.array([np.eye(num_actions)[a] for a in action_sample])
            current_q_value = tf.reduce_sum(current_q_values * action_sample_vec, axis=1)
            #-----------------#

            # compute the loss
            loss = loss_function(updated_q_value, current_q_value)

        # backpropagation
        grads = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(grads, model.trainable_variables))
    else:
        loss = 0

    # update current state and counter
    current_state = next_state
    t += 1

    # print info every 500 steps
    if t % 500 == 0:
        print(f"STEP {t} | PHASE {'observe' if t<=observe else 'train'}",
              f"| ACTION {action} | REWARD {reward} | LOSS {loss}")
```

You're now ready to play the game! Just run the cell below; do not change the code.

In [13]:
```
def playgame():

    #! DO NOT change the random seed !
    np.random.seed(4)

    model = create_q_model()

    # specify the optimizer and loss function
    optimizer = keras.optimizers.Adam(learning_rate=step_size, clipnorm=1.0)
    loss_function = keras.losses.MeanSquaredError()

    # play the game
    dql_flappy_bird(model=model, optimizer=optimizer, loss_function=loss_function)
```

In [14]:
```
playgame()
```

```
STEP 500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 1000 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 1500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 2000 | PHASE observe | ACTION 1 | REWARD 0.1 | LOSS 0
STEP 2500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 3000 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 3500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 4000 | PHASE observe | ACTION 1 | REWARD 0.1 | LOSS 0
STEP 4500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 5000 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 5500 | PHASE observe | ACTION 1 | REWARD 0.1 | LOSS 0
STEP 6000 | PHASE observe | ACTION 1 | REWARD 0.1 | LOSS 0
STEP 6500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 7000 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 7500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 8000 | PHASE observe | ACTION 1 | REWARD 0.1 | LOSS 0
STEP 8500 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 9000 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 9500 | PHASE observe | ACTION 1 | REWARD 0.1 | LOSS 0
STEP 10000 | PHASE observe | ACTION 0 | REWARD 0.1 | LOSS 0
STEP 10500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 5.519604682922363
STEP 11000 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.1459699124097824
STEP 11500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.3204263150691986
STEP 12000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.02644324116408825
STEP 12500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.04426571726799011
STEP 13000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.17679089307785034
STEP 13500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.014575055800378323
STEP 14000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.12193544209003448
STEP 14500 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.019869230687618256
STEP 15000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.035816725343465805
STEP 15500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.02135961689054966
STEP 16000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.021986842155456543
STEP 16500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.01245207991451025
STEP 17000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.030641727149486542
STEP 17500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.027269531041383743
```

```
STEP 18000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.057848379015922546
STEP 18500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.023129113018512726
STEP 19000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.04594459757208824
STEP 19500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.09974032640457153
STEP 20000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.06428693234920502
STEP 20500 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.04892557114362717
STEP 21000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.06515232473611832
STEP 21500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.04921405017375946
STEP 22000 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.06667502224445343
STEP 22500 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.04288019239902496
STEP 23000 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.15410447120666504
STEP 23500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.14670789241790771
STEP 24000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.09042476862668991
STEP 24500 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.1694066822528839
STEP 25000 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.3376889228820801
STEP 25500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.34441837668418884
STEP 26000 | PHASE train | ACTION 0 | REWARD 1 | LOSS 0.1945679932832718
STEP 26500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.5603275895118713
STEP 27000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.3755505084991455
STEP 27500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.2759215235710144
STEP 28000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.09539882838726044
STEP 28500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.17459775507450104
STEP 29000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.1618884652853012
STEP 29500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.36304396390914917
STEP 30000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 1.5559624433517456
STEP 30500 | PHASE train | ACTION 0 | REWARD 1 | LOSS 0.14109480381011963
STEP 31000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.323125422000885
STEP 31500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.2736572325229645
STEP 32000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.1740540862083435
STEP 32500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.3956851661205292
STEP 33000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.221214160323143
STEP 33500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.2073495090007782
STEP 34000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.28445377945899963
STEP 34500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.3228399157524109
STEP 35000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.748911440372467
STEP 35500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.5556262731552124
STEP 36000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.5565179586410522
STEP 36500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.12786030769348145
STEP 37000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.27960526943206787
STEP 37500 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 4.601833820343018
STEP 38000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.15723323822021484
STEP 38500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.19916746020317078
STEP 39000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.22537142038345337
STEP 39500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.16720128059387207
STEP 40000 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.22658853232860565
STEP 40500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 1.3795297145843506
STEP 41000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.17201894521713257
STEP 41500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.23382019996643066
STEP 42000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.26017969846725464
STEP 42500 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.6970758438110352
STEP 43000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.3062551021575928
STEP 43500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.20538760721683502
STEP 44000 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.12391213327646255
STEP 44500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.30556315183639526
STEP 45000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.2083311676979065
STEP 45500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.19273662567138672
STEP 46000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.39121773838996887
STEP 46500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.23889252543449402
STEP 47000 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.24404506385326385
STEP 47500 | PHASE train | ACTION 1 | REWARD 0.1 | LOSS 0.232245534658432
STEP 48000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.1129198670387268
STEP 48500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.3767557442188263
STEP 49000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.18678979575634003
STEP 49500 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.31289440393447876
STEP 50000 | PHASE train | ACTION 0 | REWARD 0.1 | LOSS 0.2217678427696228
```

4.5 Describe the training

Describe what you see by answering the following questions:

- In the early stage of training (within 2,000 steps in the *explore* phase), describe the behavior of the Flappy Bird. What do you think is the greedy policy given by the estimation of the Q-function in this stage?
- Describe what you see after roughly 5,000 training steps. Do you see any improvement? In particular, compare Flappy's behavior with their behavior in the early stages of training.
- Explain why the performance has improved, by relating to the model design such as the replay memory and the exploration.

- In the early stage of training (within 2,000 steps in the *explore* phase)

    - The bird almost randomly chooses the actions and often crashes quickly because the model has not learned much about the relationship between value and action.
    - The greedy policy in this stage is to choose the action with the highest Q value. But since the Q values for both actions are quite close to each other, the choices may be almost random.
- After roughly 5,000 training steps
    - The bird can now fly a relatively long distance before crashing, flapping just in time to avoid hitting the pipes or crashing to the ground. Its behavior improves a lot.
    - Compared to the early stage, the chosen actions are less random, because the model has started to capture some useful patterns.
- Why the performance has improved
    - Replay memory
        - The replay memory stores past experiences and the model draws random samples from this memory to train on them.
        - This can break the correlation between nearby state transitions to learn from more diverse situations.
    - Exploration
        - The model uses the exploration strategy to take random actions instead of always following the greedy policy to get the action with the maximum value.
        - This allows the model to discover new states and avoid the model from cycling in a previous situation which might not be good enough.

It takes a long time to fully train the network, so you're not required to complete the training. Here's a video showing the performance of a well trained DQN.